

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Principal Component Analysis

Compressing instead of discarding: the directions along which data varies, and what they are good for



The Challenge of High-Dimensional Data

Real-world scale

Modern datasets carry dimensionality that is hard to picture:

- an 800×600 colour image: **1 440 000 features**;
- a text "bag of words" model: **50 000+ features**;
- and every pixel, every word, is there for a reason — we cannot simply delete them.

Two critical problems

- 1. The curse of dimensionality.** In a high-dimensional space everything becomes "far apart", and distance-based models such as KNN stop working.
- 2. Multicollinearity.** The features are highly redundant: adjacent pixels of an image carry overlapping information, and correlated predictors make a regression unstable.

Feature Selection vs Feature Reduction

Feature Selection

Choose a **subset of the original variables**, by examining correlations, p-values, or with Ridge/Lasso regression.

Risk: discarding a variable throws its information away, unless it was perfectly reconstructible from the others.

Feature Reduction

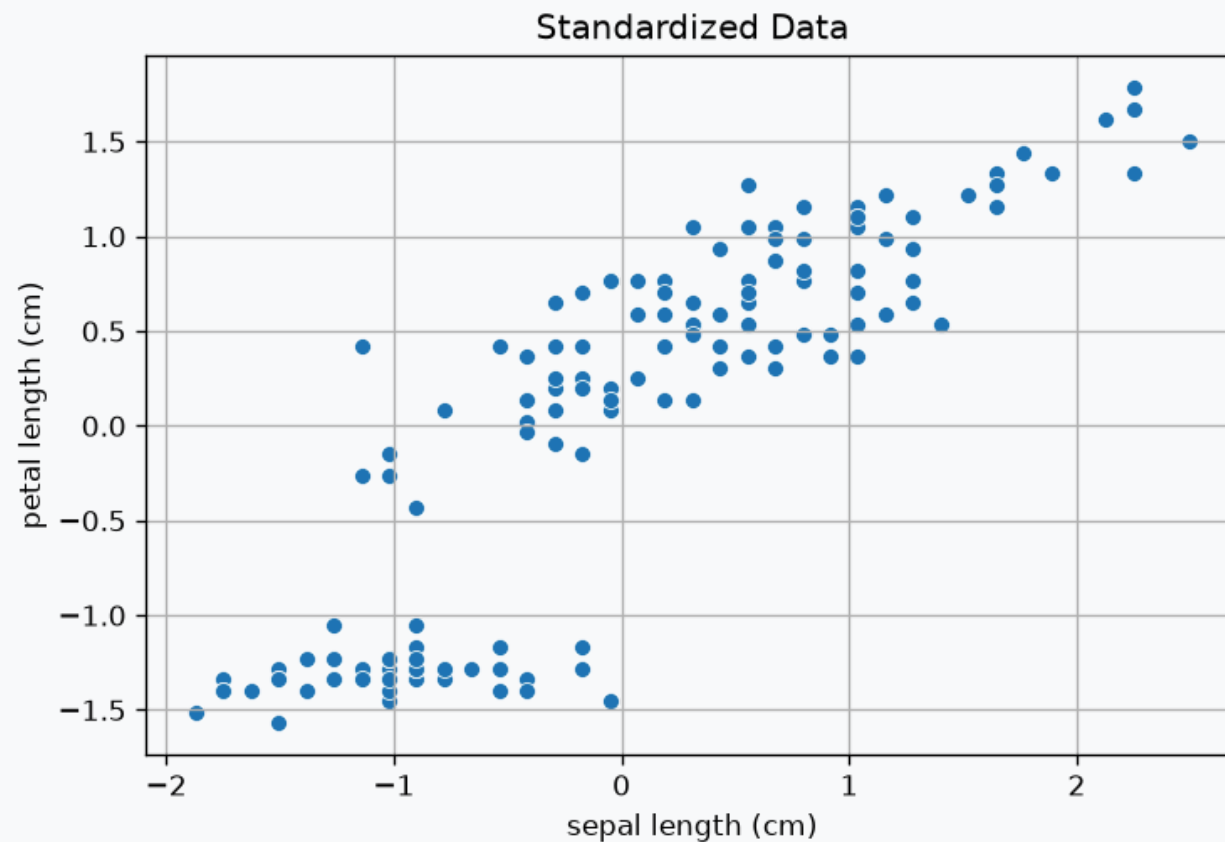
Create a **new set** of features that summarise the original ones while losing as little information as possible.

Transform the data from high dimensions into a smart, compressed representation.

When features are highly correlated, feature selection discards the redundant ones. But what if every feature matters? That is where **dimensionality reduction** becomes essential: we compress rather than discard.



A Simple Example: The Iris Dataset



Mapping 2-D data to one dimension with a linear transformation

2-D data can be mapped to a single dimension with a linear transformation:

$$z = u_1x_1 + u_2x_2 = \mathbf{u}^T \mathbf{x}$$

This maps the point (x_1, x_2) onto a line through the origin — there is no bias term — with the same direction as $\mathbf{u} = (u_1, u_2)$.

In practice we take $\mathbf{u}$ to be a unit vector:

$$\|\mathbf{u}\|_2 = 1$$

Sepal length and petal length are highly correlated — Pearson coefficient **0.87**, p-value 1.04e-47. Instead of choosing one of the two, we can project both onto a single dimension that keeps as much of the variation as possible.

- Throughout we assume the data is **standardised**, so that an axis with larger variance does not dominate the result for reasons of units alone.

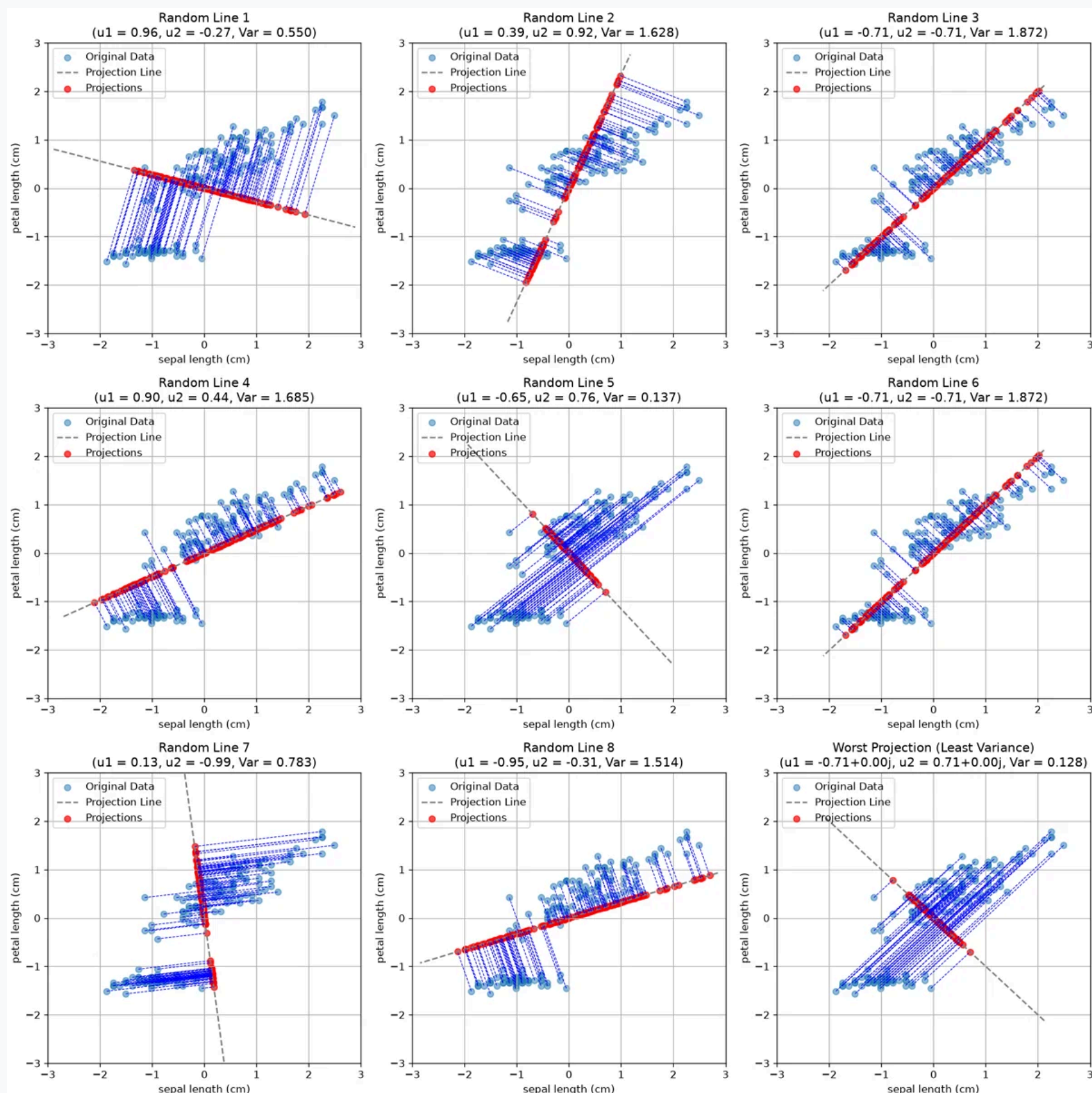


Finding the Optimal Mapping

Different mappings give different results. Some compress the points together and lose their distinctiveness; others keep them apart.

We quantify distinctiveness with the **variance of the mapped data**: higher variance means the points keep their individuality, and two distinct points do not collapse onto one once mapped.

The best mapping **maximises the variance** of the projection.



Nine directions, and the variance each one leaves: 1.872 for the diagonal that follows the cloud, 0.128 for the one perpendicular to it, where the two groups fall on top of each other.



General Mathematical Formulation

Let $\{\mathbf{x}_n\}$ be a set of N observations, $n = 1, \dots, N$, where $\mathbf{x}_n$ has dimensionality D . We define **Principal Component Analysis** as a projection of the data to $M < D$ dimensions such that the variance of the projected data is maximised.

- To avoid any influence of the individual variances along the original axes, we assume that **the data is standardised**.

Special case: $M = 1$

We first take $M = 1$: find a D -dimensional vector $\mathbf{u}_1$ such that projecting the data onto it maximises the variance. Without loss of generality we choose $\mathbf{u}_1^T \mathbf{u}_1 = 1$ — a unit vector, since only the *direction* of the projection matters.

Original data

Mean:

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$$

Covariance matrix:

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

Projected data

Projecting a point onto $\mathbf{u}_1$ is a dot product, $\mathbf{u}_1^T \mathbf{x}$, so the projected data has mean $\mathbf{u}_1^T \bar{\mathbf{x}}$ and variance

$$\frac{1}{N} \sum_{n=1}^N (\mathbf{u}_1^T \mathbf{x}_n - \mathbf{u}_1^T \bar{\mathbf{x}})^2 = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$$

The variance of the projection is a quadratic form in the covariance matrix of the data.



Maximizing Variance: An Eigenvalue Problem

Our objective is to find the projection vector $\mathbf{u}_1$ maximising the variance of the projected data, which is an optimisation problem with a constraint:

$$\arg \max \{ \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 \} \text{ subject to } \mathbf{u}_1^T \mathbf{u}_1 = 1$$

The constraint $\mathbf{u}_1^T \mathbf{u}_1 = 1$ is crucial: it makes us find a *direction* rather than a scale. Without it we could inflate the variance endlessly by choosing longer and longer vectors.

The solution of this problem leads directly to **eigenvalues and eigenvectors**: the variance is maximised when

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \lambda_1 \quad \text{or equivalently} \quad \mathbf{S} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1$$

Here $\mathbf{u}_1$ is an **eigenvector** of the covariance matrix $\mathbf{S}$ and λ_1 is its **eigenvalue**, which is the variance captured along that direction.

Since we want to maximise $\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \lambda_1$, in practice we choose as $\mathbf{u}_1$ the **eigenvector corresponding to the largest eigenvalue**.

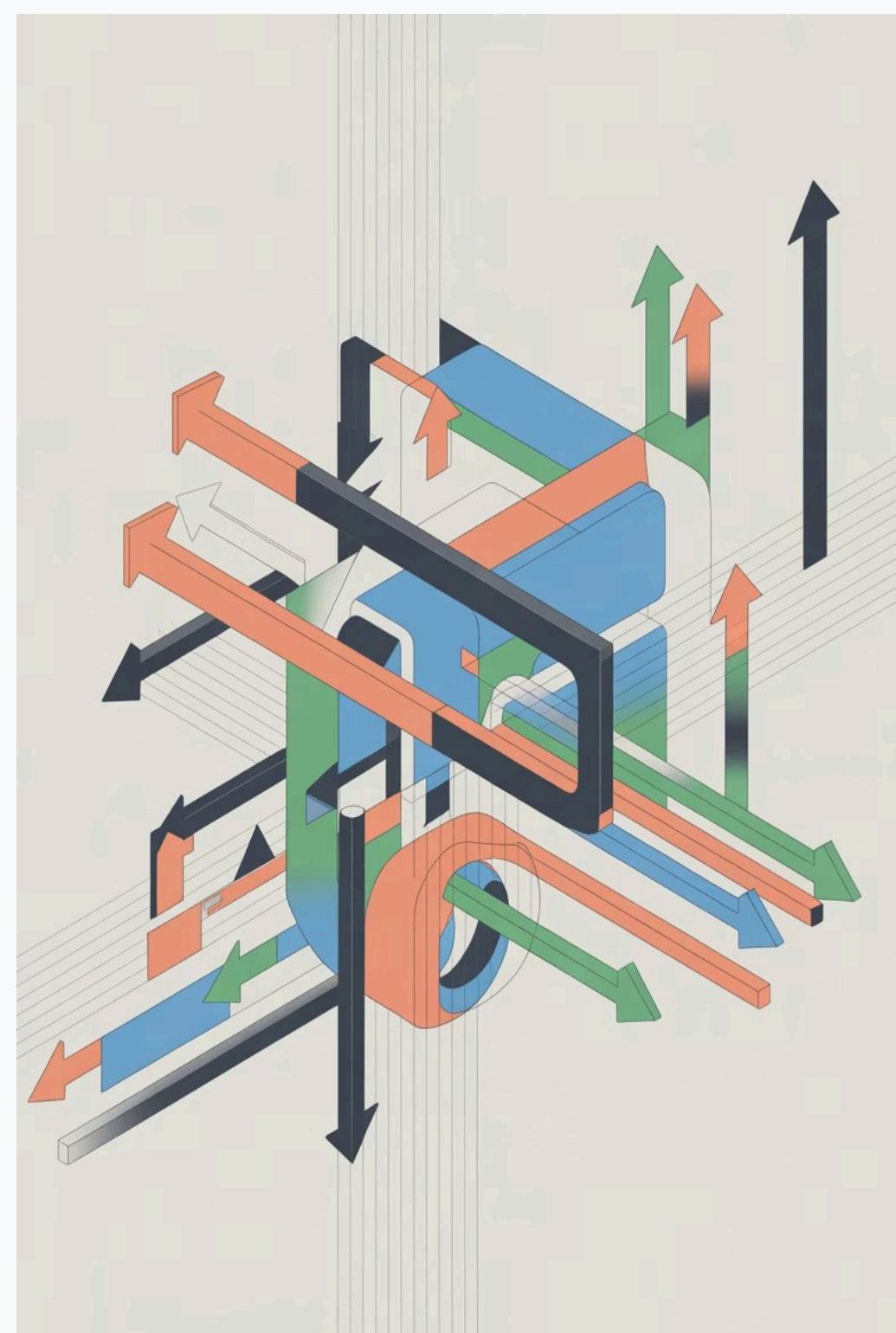


The Elegant Solution: Eigenvectors

The vector $\mathbf{u}_1$ that maximises the variance of the projected data is the **first principal component** of the data. It is the eigenvector of the covariance matrix $\mathbf{S}$ with the largest eigenvalue λ_1 .

We can proceed iteratively to find the other components. To build a new, orthogonal reference system, we choose the second principal component $\mathbf{u}_2$ to be orthogonal to the first ($\mathbf{u}_2 \perp \mathbf{u}_1$): it is the vector that maximises the variance of the projected data **among all vectors orthogonal to $\mathbf{u}_1$** .

In practice, it can be shown that the M principal components are the M eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_M$ of the covariance matrix $\mathbf{S}$ corresponding to the M largest eigenvalues $\lambda_1, \dots, \lambda_M$.



Matrix Formulation

To see PCA in matrix form, we define the data matrix $\mathbf{X}$ and the transformation matrix $\mathbf{W}$:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_N \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ & \dots & & \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{pmatrix} \quad \mathbf{W} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \dots \\ \mathbf{u}_M \end{pmatrix} = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1D} \\ u_{21} & u_{22} & \dots & u_{2D} \\ & \dots & & \\ u_{M1} & u_{M2} & \dots & u_{MD} \end{pmatrix}$$

$\mathbf{X}$ is the $[N \times D]$ data matrix: N observations, D original features.

$\mathbf{W}$ is an $[M \times D]$ matrix, each row a principal component of $\mathbf{S}$.

The whole dataset is transformed in one product:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T$$

The projected data $\mathbf{Z}$ is an $[N \times M]$ matrix, each row an observation in the reduced M -dimensional PCA space. These new dimensions are called **latent variables**: they uncover underlying patterns and their values are not directly observed.



A Computational Note

In practice we compute all D eigenvectors of the covariance matrix $\mathbf{S}$ and arrange them into a full transformation matrix $\mathbf{W}_{all}$, sorted by eigenvalue in descending order:

$$\mathbf{W}_{all} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \dots \\ \mathbf{u}_D \end{pmatrix}$$

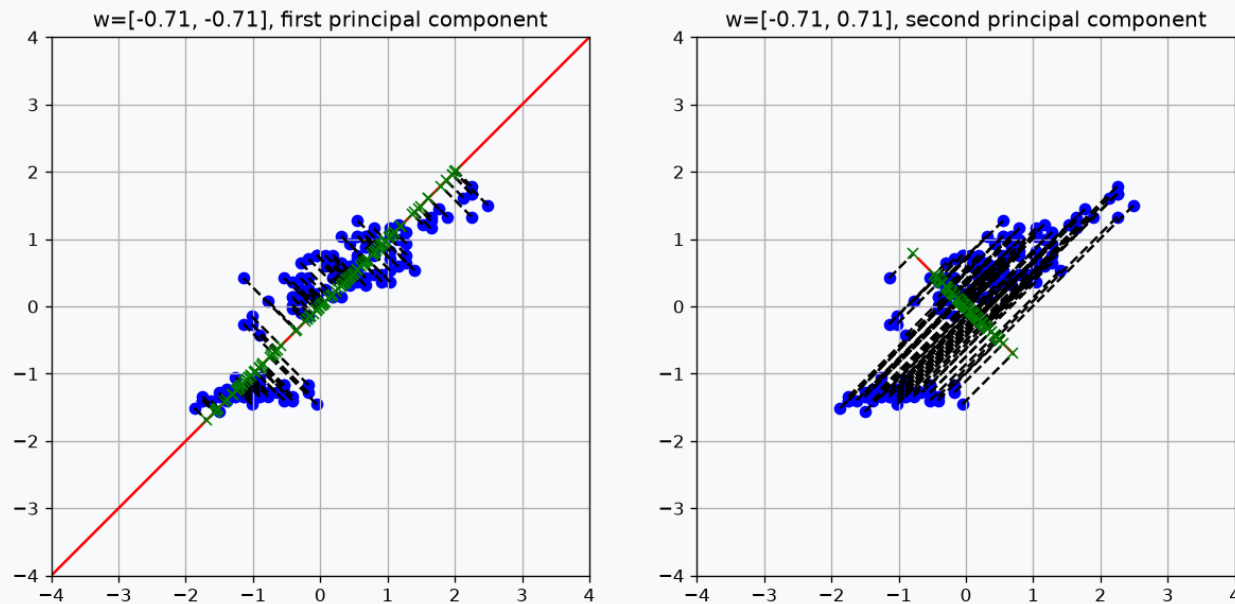
We then **truncate** $\mathbf{W}_{all}$, keeping the top M eigenvectors, which form the transformation matrix $\mathbf{W}$ we want. To reduce to $M = 2$ dimensions, for instance:

$$\mathbf{W} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \end{pmatrix}$$

One decomposition serves every choice of M : changing the number of components is slicing rows off a matrix we already have.



Back to Our Iris Example



The two plots show the points mapped onto the line of each component. The first keeps the spread — the eigenvalue 1.884 — while the second collapses almost everything onto one spot, which is what an eigenvalue of 0.129 means.

Covariance matrix

```
[[1.007 0.878]
 [0.878 1.007]]
```

The high correlation, 0.878, confirms the redundancy between the two features.

Eigenvalues and eigenvectors

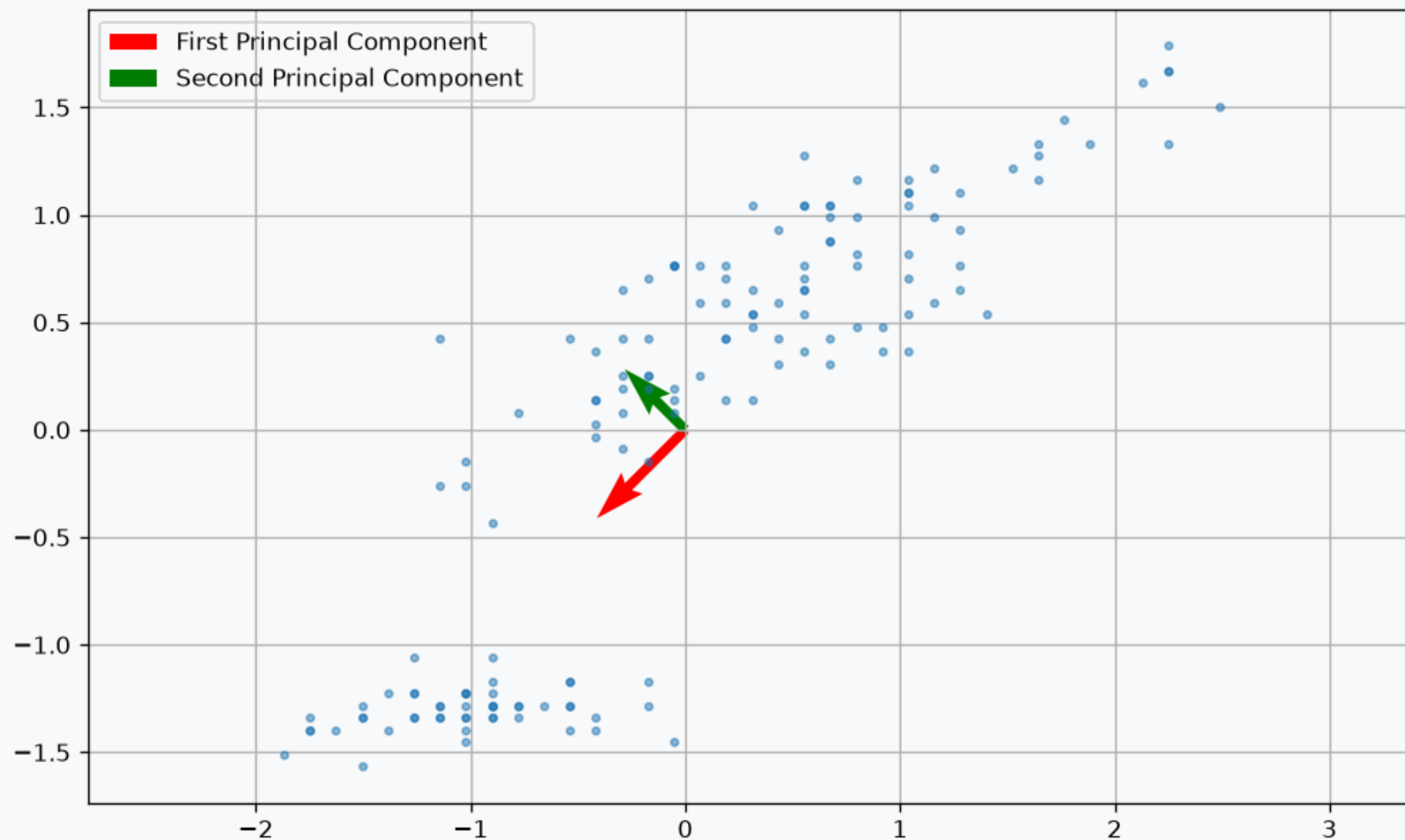
```
λ: [1.884, 0.129]
u: [[-0.707, -0.707]
     [-0.707, 0.707]]
```

Sorted by eigenvalue, the eigenvectors are the two principal components: the first along the diagonal of the cloud, the second across it.



Visualizing Principal Components

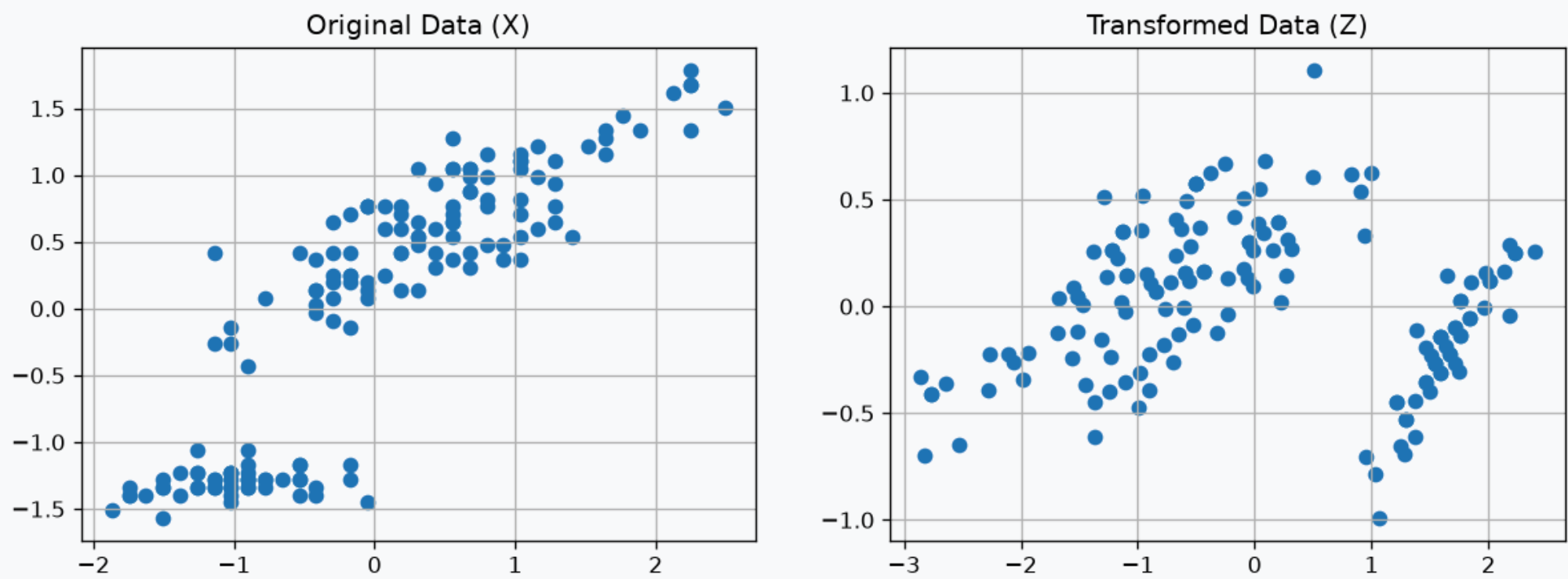
The two principal components identify **orthogonal directions of maximum variation**. The first component, the longer arrow, captures most of the variance; the second captures what is left, perpendicular to the first.



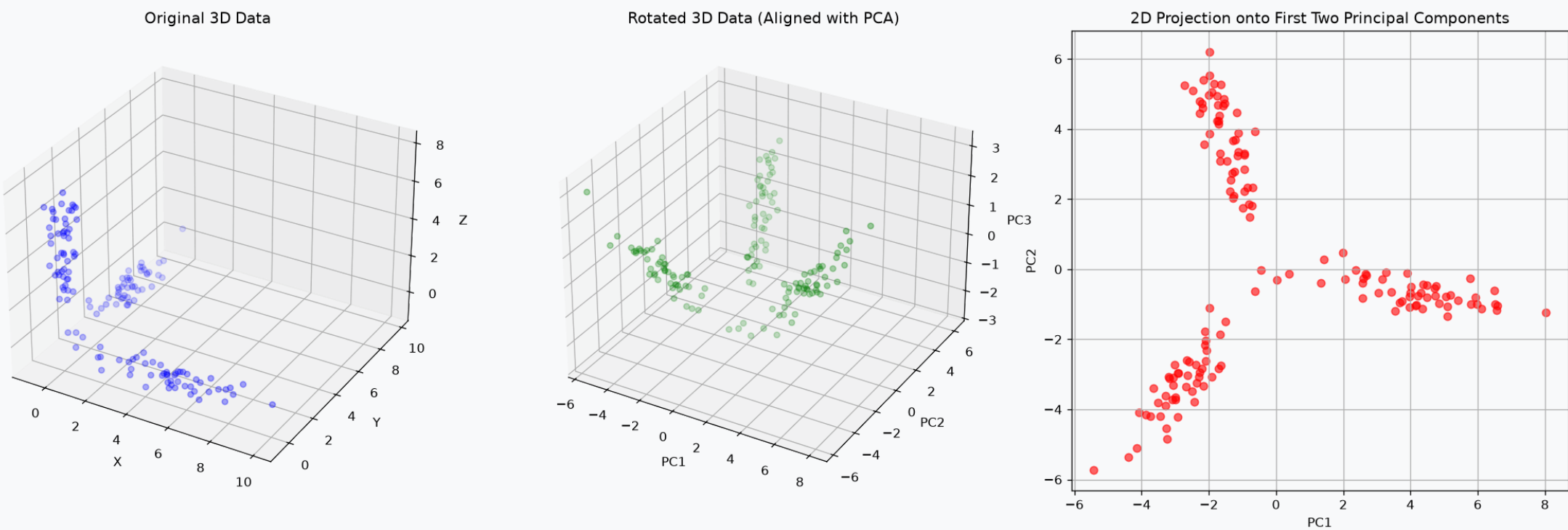
The arrows are drawn from the mean of the standardised data, with length proportional to the square root of their eigenvalue: 1.37 against 0.36. Nothing in the data distinguishes the direction of an arrow from its opposite — an eigenvector is defined up to sign.

PCA as Data Rotation

If we set $M = D$, PCA becomes a **rotation** of the data around the origin — recall that the data is standardised, so it is already centred.



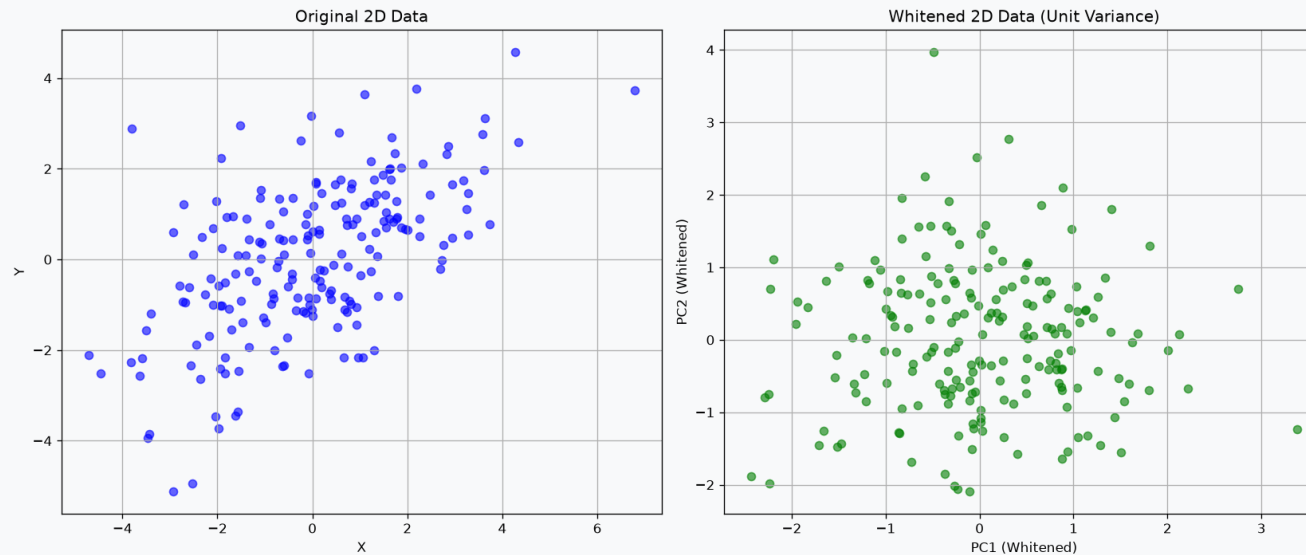
When $M < D$ we can see PCA as a two-step process:



In three dimensions: the original data, the same data rotated so that its directions of variation are the axes, and the 2-D projection onto the first two components.



PCA for Data Whitening



Decorrelation property

PCA transforms data with covariance matrix Σ into new variables with a **diagonal covariance matrix** Σ' : the new variables are completely uncorrelated.

Why it works: we rotated the data so that the axes align with the main directions of variation, and we built the components to be orthogonal.

Original covariance

```
array([[3.60543087, 1.84308349],  
       [1.84308349, 2.83596216]])
```

Covariance of the transformed data

```
array([[ 1.00000000e+00, -1.40945288e-16],  
       [-1.40945288e-16,  1.00000000e+00]])
```

Application: whitening is valuable for linear regression with correlated variables — preprocessing with PCA removes multicollinearity and stabilises the fit, at the cost of interpretability.



Choosing the Number of Components

How many components M should we keep? The answer depends on how much information we need to retain, and we measure that through the **cumulative explained variance**. Let $Var(z_i)$ be the variance of the data along the z_i component. The total variance is the sum of the variances:

$$V = var(z_1) + var(z_2) + \dots + var(z_D)$$

and the fraction of variance explained by component n is

$$\frac{var(z_n)}{V(D)}$$

Taking the whole Fisher Iris dataset — four features — the principal components and the covariance matrix of the transformed data are:

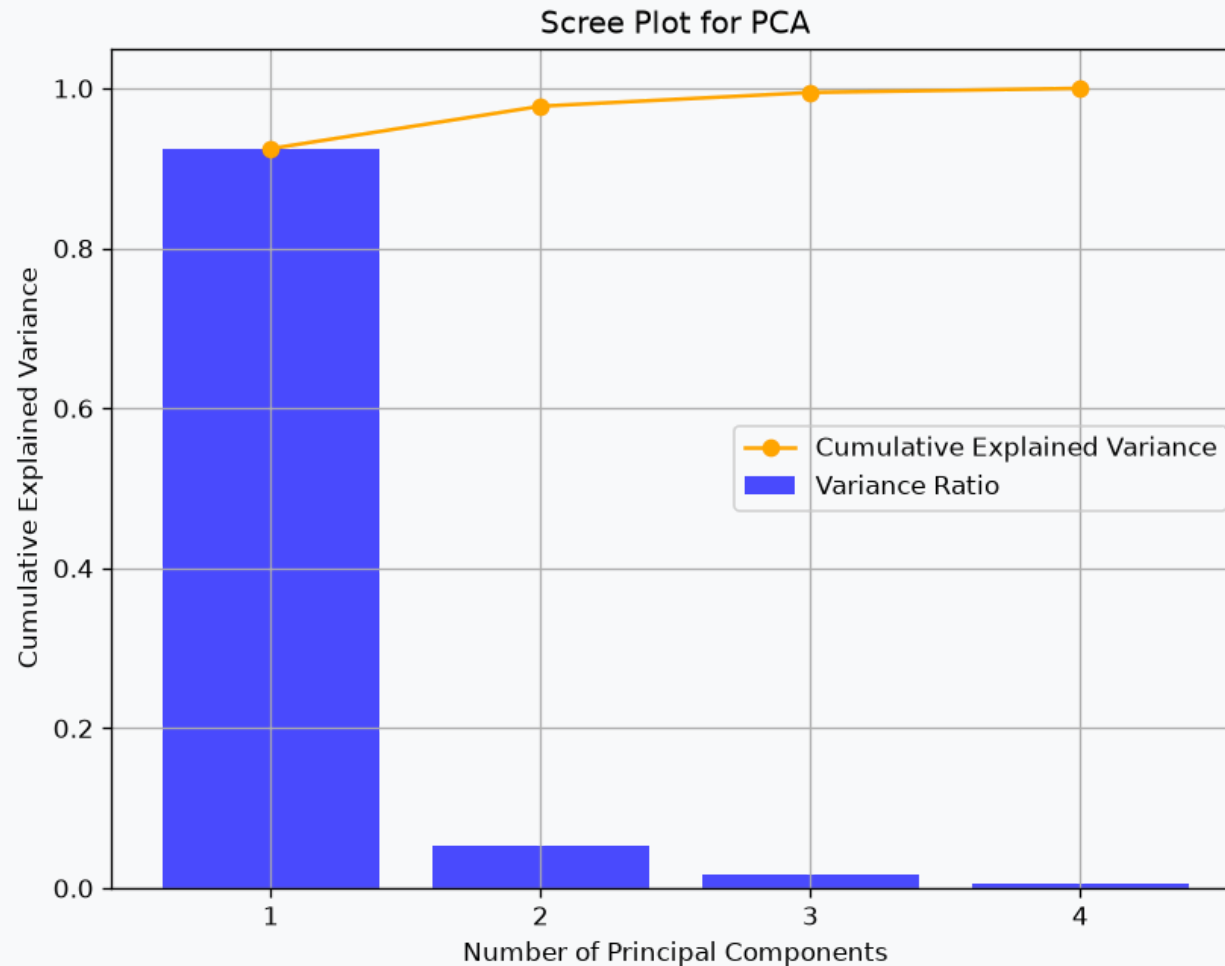
	sepal length	sepal width	petal length	petal width
z1	0.361387	0.656589	−0.582030	0.315487
z2	−0.084523	0.730161	0.597911	−0.319723
z3	0.856671	−0.173373	0.076236	−0.479839
z4	0.358289	−0.075481	0.545831	0.753657

	z1	z2	z3	z4
z1	4.23	0.00	0.00	0.00
z2	0.00	0.24	−0.00	0.00
z3	0.00	−0.00	0.08	−0.00
z4	0.00	0.00	−0.00	0.02

The off-diagonal zeros are the decorrelation property again; the diagonal is the variance each component carries.



Choosing the Number of Components: The Scree Plot



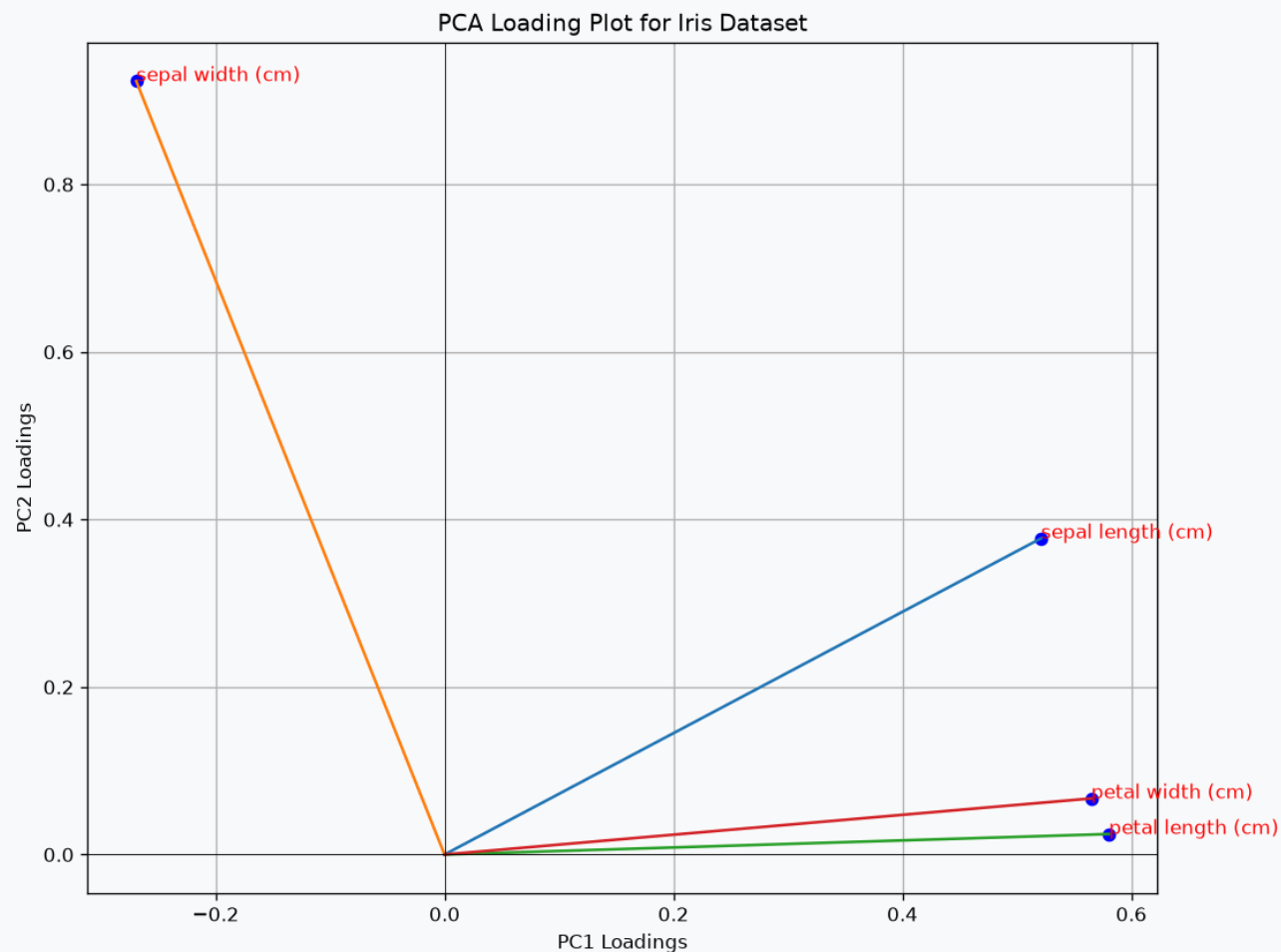
In practice it is common to visualise the explained variance ratio in a **scree plot**: the bars are the variance each component explains, the line is the cumulative total.

On Iris, the first component alone explains about **92%** of the variance, and two components take us past 97%. Keeping two of the four features loses almost nothing — which is why the four-dimensional dataset can be drawn on a page.

The usual reading is either a threshold ("keep enough components to reach 95%") or the elbow of the curve, which is the same heuristic we used to choose K for K-means.



Interpreting Components: Load Plots



Load plots place the **original variables** on the plane of the first two components. The coordinates are the "loadings", and a loading with a large absolute value means that variable matters more in determining that component.

Here petal length, petal width and sepal length all point right, close together: they are the correlated group that the first component summarises. Sepal width points up and to the left, which is why it needs the second component.

Clustered variables

Variables appearing together are positively correlated; opposite sides indicate negative correlation.

Distance from origin

Variables farther from the centre have higher loadings: they weigh more in the components.

Feature selection guide

Identify groups of correlated variables and keep one representative of each, to reduce redundancy.



Laboratory: Applications of PCA

Dimensionality reduction and visualization

Simplify a complex dataset by reducing the number of features while retaining most of the information, which makes high-dimensional data visible.

Principal component regression

Overcome multicollinearity in a linear model by using uncorrelated principal components as predictors, which stabilises the fit.

Data compression

Reduce storage and transmission by representing data with fewer principal components, preserving the essential variance.

Eigenfaces

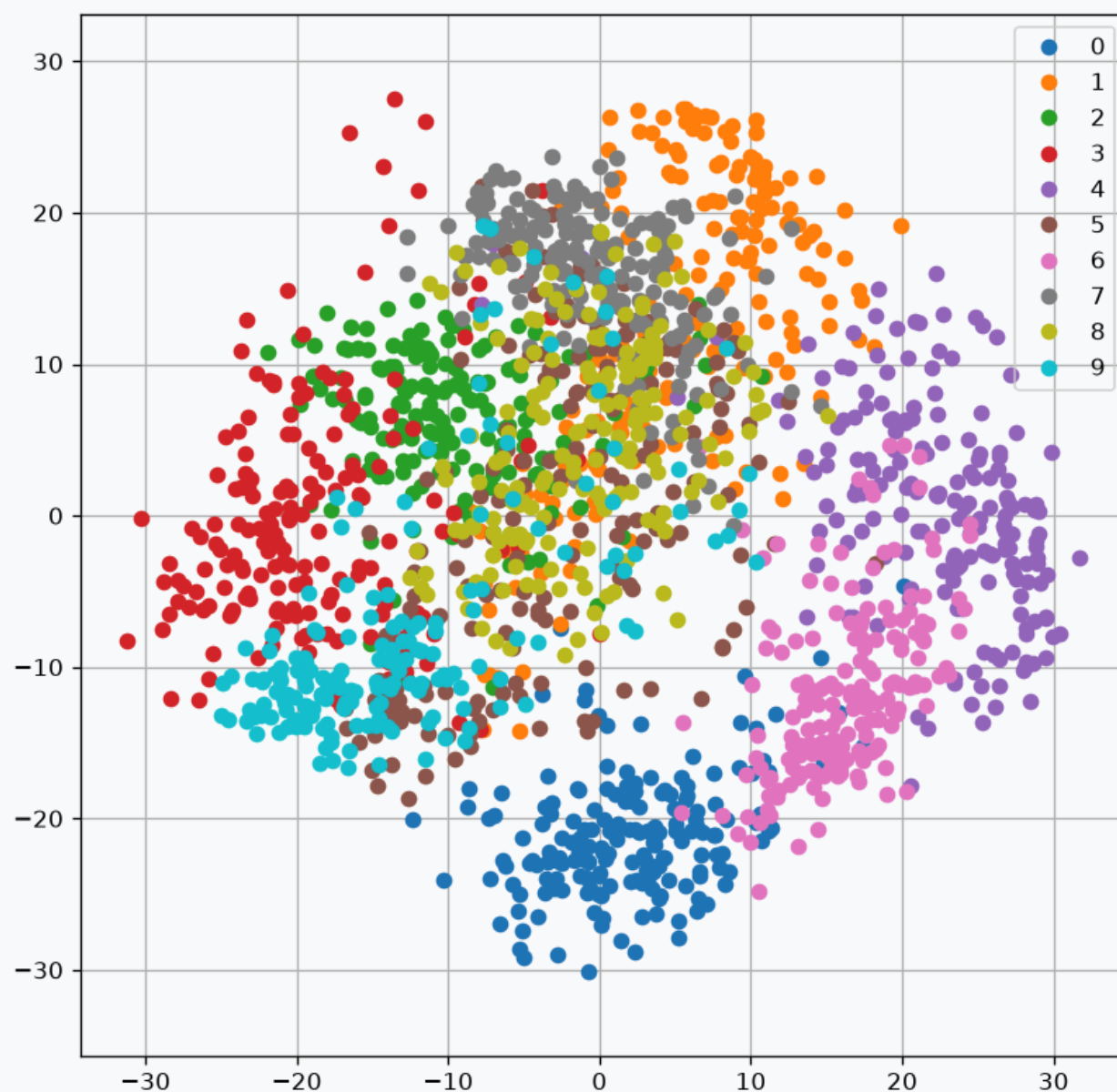
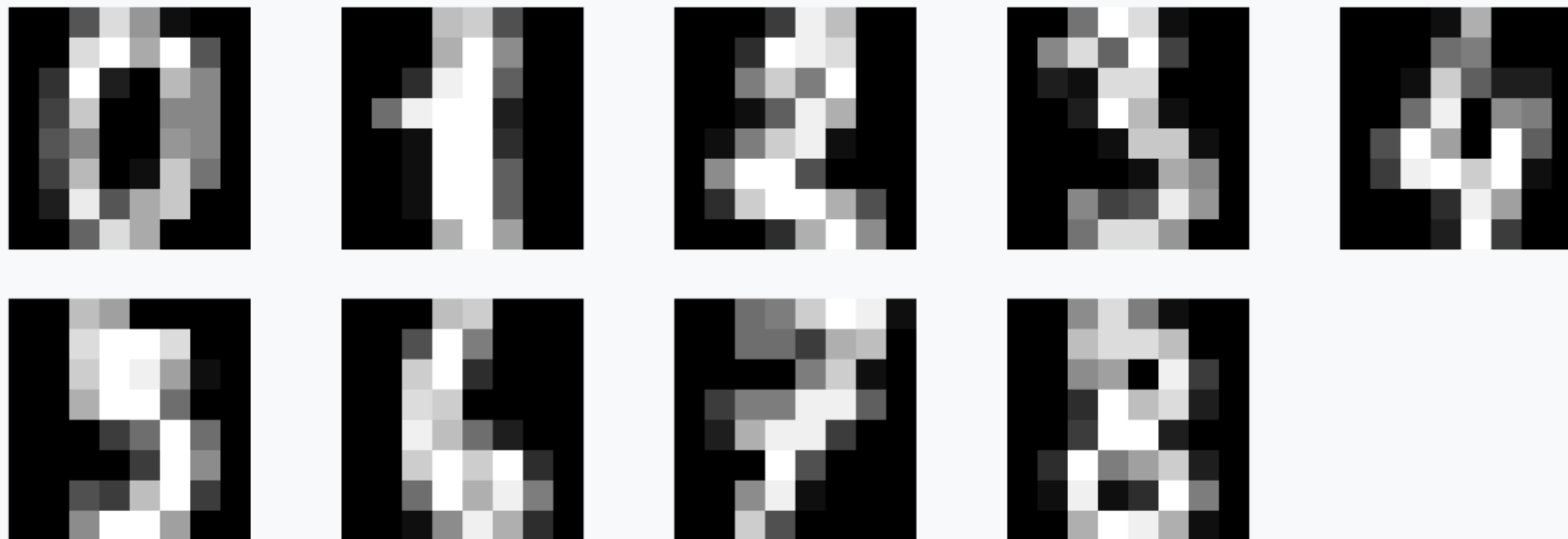
The classic computer vision application: faces represented as combinations of principal components extracted from a set of face images.



Lab: Dimensionality Reduction and Visualization

When $D = 2$ a scatterplot shows how data distributes; when D is a little larger a pairplot still works; when D is very large neither does. PCA gives us a way to look at the data anyway: project to the first two components and plot that.

The **DIGITS** dataset holds 8×8 images of handwritten digits, so each image is a point in **64 dimensions** — one variable per pixel.



Projected onto two components and coloured by their true label, the digits arrange themselves into groups: zeros in one region, ones in another, with the confusable pairs — 3, 5 and 8 — overlapping where we would expect them to.

Nothing in the projection knew about the labels. The structure was in the pixels, and two of sixty-four directions were enough to make it visible.

Lab: Principal Component Regression

PCA is also useful when we fit a linear regressor on data with **multicollinearity**. Take the `auto_mpg` dataset and regress `mpg` on all seven predictors: several of them come out with large p-values, because they are correlated with the others — displacement, cylinders, horsepower and weight all say much the same thing about a car.

Principal component regression (PCR) computes PCA **on the independent variables** first, then fits the regression on the components. Keeping $M = 4$ components:

$$mpg \sim z_1 + z_2 + z_3 + z_4$$

The predictors are now uncorrelated by construction, so the fit is stable and each coefficient is estimated cleanly.

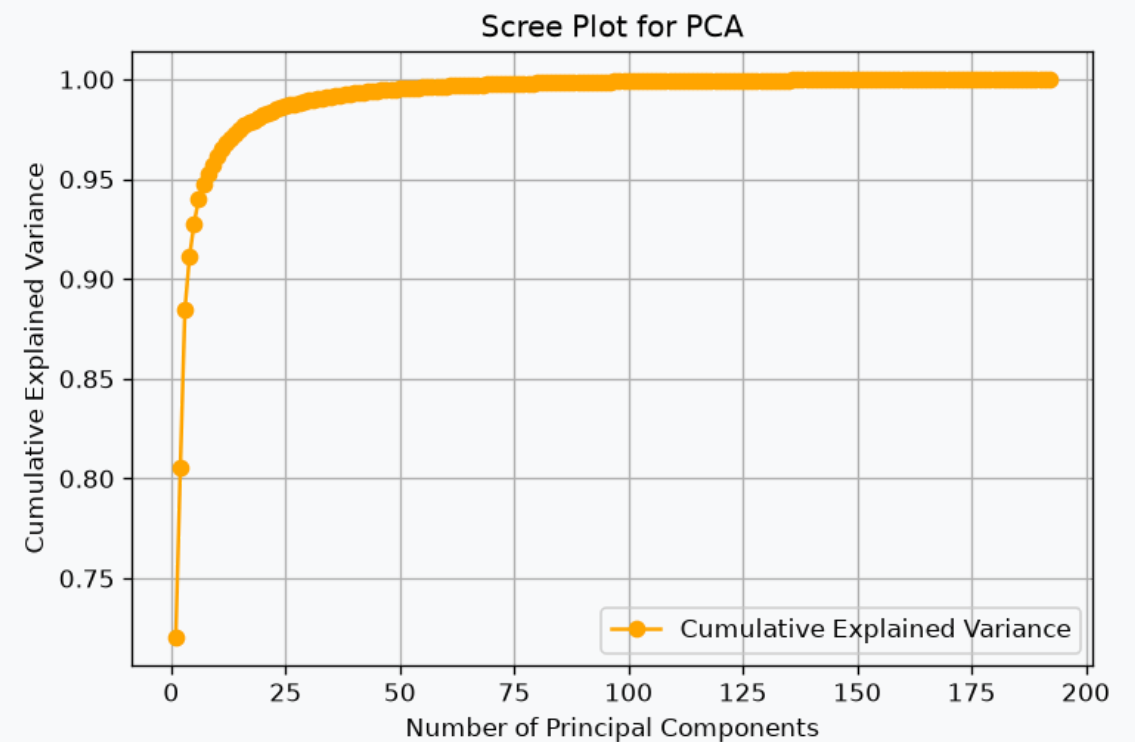
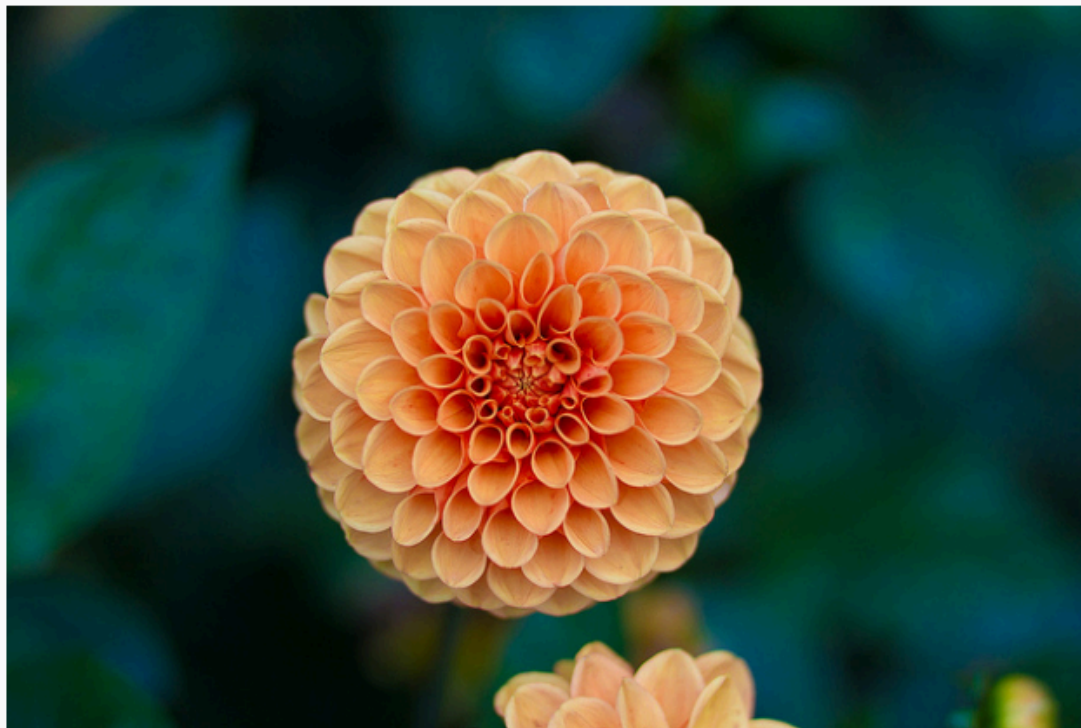
- The price is interpretability: z_1 is a mixture of displacement, weight and horsepower, and "one unit of z_1 " has no meaning a mechanic would recognise. PCR is a tool for **predictive** analysis, not for explanation.



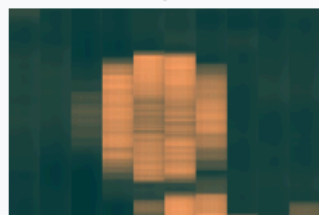
Lab: Data Compression

An image is high-dimensional data: one dimension per pixel value. The photograph below is 427×640 in colour — **819 840 dimensions** — and much of that is redundant.

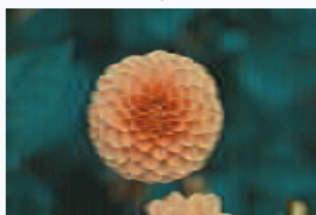
The method: cut the image into fixed-size blocks (8×8), treat the blocks as a population of observations, and run PCA on them.



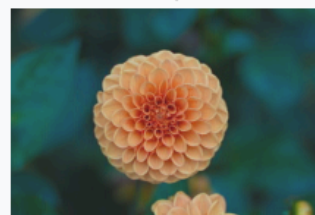
M=1 Comp: 99.48%



M=8 Comp: 95.83%



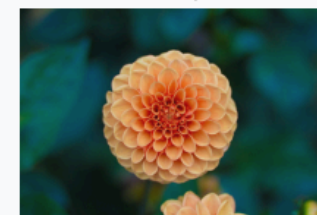
M=32 Comp: 83.33%



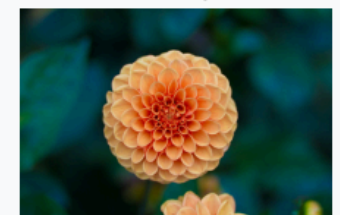
M=64 Comp: 66.67%



M=128 Comp: 33.33%



M=192 Comp: 0.00%

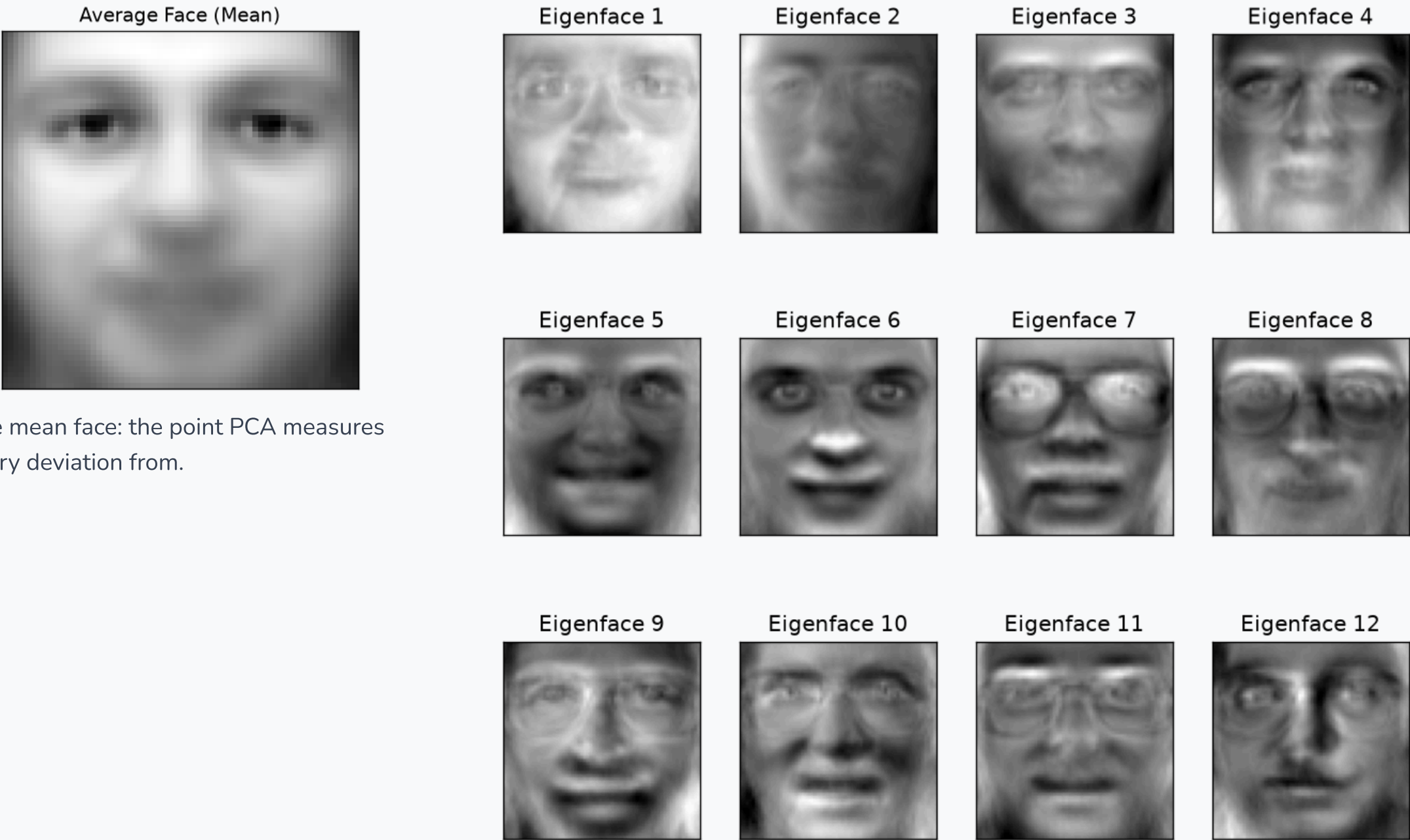


The reconstruction at $M = 1, 8, 32, 64, 128$ and 192 components, with the space saved above each: at 32 components the photograph is recognisable and **83.33%** of the storage is gone; by 64 the difference from the original is hard to see.

Lab: Eigenfaces

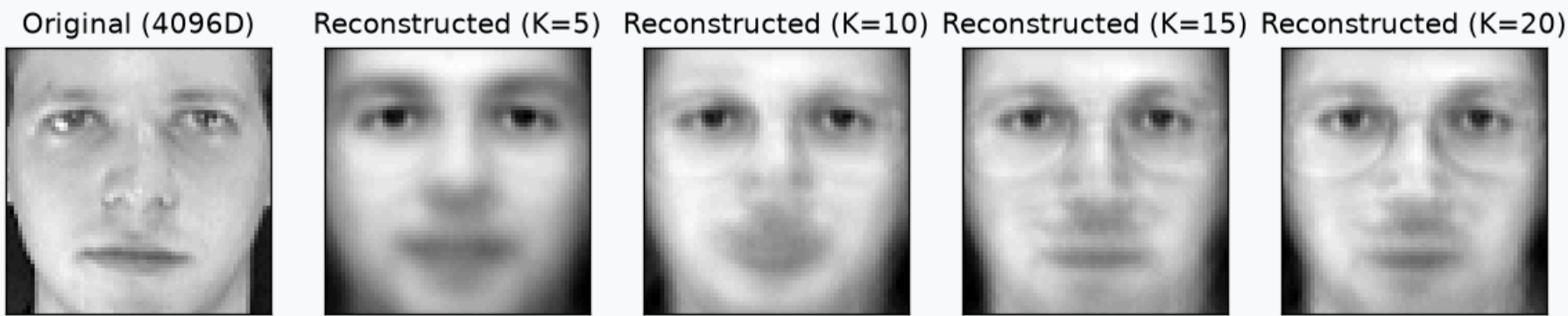
Applied to a dataset of faces, the principal components are no longer abstract directions — they can be drawn as **ghostly faces** of their own, the "eigenfaces". Each one is a fundamental component of a human face: lighting from the left, the shape of a nose, a smile against a frown.

The Olivetti dataset holds **400 faces of 64 × 64 pixels**, that is **4 096 features** each.



The mean face: the point PCA measures every deviation from.

PCA for Image Compression and Reconstruction



A face compressed from 4 096 numbers to 5, 10, 15 and 20, then reconstructed. Twenty numbers are enough to recognise the person.

Lab: Are Eigenface Features Actually Better?

PCA compresses a 4 096-pixel image into a short vector. But is that vector a *better feature set* for a model? A bake-off answers it: the same KNN classifier for face recognition, on 280 training and 120 test images, in two different feature spaces.

79.17%

KNN on raw pixels

4 096 features, standardised

83.33%

KNN on PCA features

20 components, whitened

PCA did not merely *compress* the data, it **improved** it. The low-dimensional space is cleaner: the directions that carried mostly noise are gone, so the distances the classifier relies on mean more than they did in the pixel space.

This is the lesson of chapter 17 again, from the other side. There the representation came from a pre-trained network; here it is computed from the dataset itself, with no labels and no training.



Conclusions and Next Steps



We Have Explored:

The challenge of high-dimensional data

Feature selection against feature reduction

PCA, and the variance it maximises

The eigenvalue problem behind it

The matrix formulation, and whitening

Choosing the number of components

Loading plots

Visualisation, PCR, compression, eigenfaces

- In the next lecture we look at Fisher's linear discriminant, where the same machinery is used with the labels in hand.

References

- Parts of Chapter 12 of [1]

[1] Bishop, Christopher M. *Pattern recognition and machine learning*. Springer, 2006.

- Lecture notes: [Chapter 20 — Principal component analysis](#)

